Hito 1

Adquisición y núcleo RTOS

PARTE A — *EL SENSOR Y SU BUS*

PASO A.1 — LA CONEXIÓN FÍSICA.

RETO 1.1 — CONEXIÓN FÍSICA DEL SENSOR

1.1.1). Completar la Tabla 1.1 e indicar qué periférico SPI se usa.

Señal	Pin MCU	Función (AF o GPIO)	Papel	Pág
SCK	30	PA5	Reloj de bus	26
MISO	31	PA6	Datos del sensor hacia el MCU	26
MOSI	32	PA7	Datos del MCU hacia el sensor	26
CS	2	PE3	Selección de Chip	30

1.1.2. La línea CS, ¿la gobierna el periférico SPI (NSS hardware) o un GPIO? ¿A qué nivel está activa? ¿Qué consecuencia tiene para el driver?

La línea CS está controlada por el maestro (el micro) que lo maneja como un GPIO (LIS3DSH, pág. 25). Según la información que aparece en la tabla 2 de pág. 11 del LIS3DSH, CS es activo a bajas, se comunica por SPI con CS=0. La consecuencia que tiene manejar CS por GPIO es que el driver debe conmutar el pin manualmente a nivel bajo antes de la transacción y restaurarlo a alto al finalizar.

Al finalizar la transacción, CS debe volver a nivel alto. El driver debe garantizar esta secuencia incluso cuando se produzca un error en la transferencia.

PASO A.2 — LOS REGISTROS DEL SENSOR.

RETO 1.2 — CONFIGURACIÓN DEL SENSOR

1.2.1. Completar la Tabla 1.2, incluido el orden y el formato de los bytes de cada eje (con o sin signo, número de bits).

Registro	Dir (hex)	Uso en MIV-RT
WHO_AM_I	0x0F	Identificación, solo lectura: debe devolver 0x3F (firma

		del LIS3DSH). Es la prueba de vida del bus.
CTRL_REG4	0x20	Los primeros 4 bits MSB sirven para configurar la velocidad de datos de salida o power down. El bit 5 BDU nos permite decidir entre actualizar continuamente (0) o esperar a leer MSB y LSB para actualizar (1)
CTRL_REG5	0x24	Ancho de banda del filtro, fondo de escala, autotest e interfaz SPI.
CTRL_REG6	0x25	Incluye ADD_INC, para incrementar automáticamente la dirección en accesos múltiples.
OUT_X_L - OUT_*Z*_*H*	0x2D	Seis bytes de aceleración correspondientes a los tres ejes.

Cada eje se reconstruye a partir de dos bytes, bajo y alto, formando un valor de 16 bits con signo.

1.2.2. Elegir y justificar, a partir de los requisitos del modelo: la frecuencia de muestreo del snsor (ODR) frente al periodo de la tarea Sensor (RF-1); el fondo de escala (FS) para medir inclinación; y si conviene activar BDU (Block Data Update). Anotar la sensibilidad (mg/LSB) del FS elegido: se usa en la Parte B.

Se ha decidido seleccionar una frecuencia de muestreo de 100Hz para cumplir con el requerimiento de que la placa lea el acelerómetro cada 10ms, garantizando un único valor de lectura para cada paquete de datos.

BDU se coloca en estado alto "1", para así congelar las actualizaciones de registros de salida mientras se leen los LSB y MSB de cada eje, evitando así leer datos incoherentes de distintos instantes de tiempo.

1.2.3. Calcular el byte exacto que hay que escribir en CTRL_REG4 y en CTRL_REG5, mostrando de dónde sale cada bit.

CTRL_REG4:

ODR3	ODR2	ODR1	ODR0	BDU	ZEN	YEN	XEN

0	1	1	0	1	1	1	1

ODR[3:0] se selecciona en 0110 para ajustar el ratio de datos en 100Hz y así verificar el periodo de la tarea (10ms)

Z,Y,X se seleccionan en alto para habilitar los respectivos ejes de medición.

CTRL_REG5:

BW2	BW1	FSCALE2	FSCALE1	FSCALE0	ST2	ST1	SIM
1	1	0	0	0	0	0	0

BW selecciona el ancho de banda del filtro anti-aliasing el cual se configura a 50Hz, para que de esta manera se cumpla el criterio de Nyquist.

PASO A.3 — EL PROTOCOLO SPI.

RETO 1.3 — PROTOCOLO DE LECTURA Y ESCRITURA

1.3.1. ¿Cómo se distingue en el primer byte una lectura de una escritura? ¿Cuántos bytes viajan en total para leer los tres ejes de una vez, y qué permite hacerlo en una sola transacción?

Leyendo el primer bit se distingue una lectura (1) de una escritura (0). En total viajan 7 bytes para leer los tres ejes. Uno de dirección y dos de datos por cada eje. Podemos hacerlo en una sola transacción añadiendo 8 ciclos de reloj por cada registro extra (pag27 LIS3DSH).

1.3.2. Determinar el modo SPI (polaridad CPOL y fase CPHA) a partir del cronograma del datasheet y anotar dónde se configura en CubeMX.

(pag 25)

Para una lectura de seis bytes, la transacción contiene siete bytes transferidos por el maestro, uno de dirección y seis para obtener los datos.

Modo SPI 3: CPOL = 1 y CPHA = 1.

el acceso múltiple necesita que la dirección avance entre registros. El proyecto configura CTRL_REG6 = 0x10, lo que activa ADD_INC; falta verificar que la trama SPI implementada y la configuración efectiva permiten leer los seis registros consecutivos correctamente. La descripción de ADD_INC se encuentra en el datasheet.

1.3.3. Mientras el maestro envía el byte de dirección, también recibe un byte. ¿Qué contiene? ¿Qué implica para el driver?

Ese byte información espuria, ya que durante los primeros 8 ciclos de reloj el acelerómetro aún no sabe qué registro se le está solicitando. Por tanto, el driver debe descartarlo.

PASO A.4 — EL RELOJ DEL BUS.

Reto 1.4 — Prescaler y tiempo de transacción

1.4.1. Obtener fAPB del bus del que cuelga el periférico y el fSCK máximo del LIS3DSH. Comprobar qué prescaler deja CubeMX por defecto y si respeta ese límite.

El fAPB es APB2=84MHz (pag 66 manual) y fmax=10MHz (tabla 5 pag 14)

El que deja por defecto CubeMX es 2, pero no respeta el límite máximo establecido para el bus SPI.

1.4.2. Elegir el prescaler (¿cuál es el menor valor válido?) y calcular tSPI para leer los tres ejes. Expresarlo como porcentaje del C estimado de Sensor en el modelo.

- Prescaler 2: 42 MHz, por encima del máximo de 10 MHz del sensor.
- Prescaler 16: 5,25 MHz, dentro del límite.

Elegimos el prescaler de 16, obteniendo un clock de 5.25 MHz.

$$t_{spi} = \frac{bits\ totales}{f_{sck}} = \frac{7bytes \cdot \frac{8bits}{byte}}{\frac{84MHz}{16}} = 10.66\,\mu s$$

T	10ms	Periodo de la tarea
C	0.8ms	Duración de la tarea
t_{spi}	10.66 µs	Duración de la comunicación SPI

$$Porcentaje = \frac{t_{spi}}{C} \cdot 100 = 1.3\%$$

1.4.3. Con ese resultado, ¿es razonable leer por polling o haría falta DMA?

La transferencia SPI apenas dura un 1.3% del tiempo de cómputo de la tarea Sensor. Por tanto, es razonable usar polling: la CPU se quedará esperando durante esos 10,7 µs sin hacer nada más, pero es tan poco tiempo que no merece la pena evitarlo.

PASO A.5 — EL DRIVER.

Reto 1.5 — Driver y prueba de vida.

1.5.1). Completar el listado 1.1 y llamarlo desde main: who_am_i debe devolver 0x3f de forma estable.

El Driver se ha implementado haciendo uso de tres funciones. Por un lado, con *lis_read* se leen los n registros consecutivos en una única transacción full-duplex de n + 1 bytes. El primer byte transmitido tx[0] contiene la dirección con el bit 7 a 1 (0x80 | reg), y el resto son bytes de relleno que mantienen el reloj activo para que el sensor pueda responder. Del vector recibido se descarta el primer byte (rx[0]), que, como comentamos en la *Sección A.3*, llega mientras se está transmitiendo la dirección y por tanto no contiene datos válidos, así se obtienen los n datos útiles. Por ejemplo, para leer los tres ejes (n = 6) viajan 7 bytes: uno de dirección y seis de datos.

Por otro lado, con *lis_write* se envían un byte de dirección (tx[0]) con el bit 7 a 0 y otro con lo datos (tx[1]) que se desean escribir.

Por último, con *lis_init,* se lee el registro WHO_AM_I y solo si devuelve el valor esperado, escribe CTRL_REG4 = 0x6F, CTRL_REG5 = 0xC0 y CTRL_REG6 = 0x10. Devuelve 0 si todo ha ido bien y 1 en caso contrario.

Se usa una única llamada a HAL_SPI_TransmitReceive en lugar de Transmit + Receive, tal y como se indica en el guion. Esto permite reflejar lo que ocurre en el bus, ya que por cada byte que sale entra otro, y partir la transacción dejaría un byte recibido huérfano.

Expression	Type	Value
(x)= who	volatile uint8_t	0x3f
(x)= init_err	volatile uint8_t	0x0

Figura 1 *– WHO_AM_I devuelve 0x3F y init_err=0.*

1.5.2). Tras escribir CTRL_REG4 y CTRL_REG5, leerlos de vuelta y comprobar que contienen lo escrito.

Tras ejecutar *lis_init(),* que escribe los registros de control, se vuelven a leer CTRL_REG4, CTRL_REG5 y CTRL_REG6 con *lis_read* y se guardan en las variables r4, r5 y r6. Se obtiene CTRL_REG4 = 0x6F, CTRL_REG5 = 0xC0 y CTRL_REG6 = 0x10, que coinciden con los valores escritos (Figura 2). Esto confirma que las escrituras llegan correctamente al sensor y que queda configurado a 100 Hz, con BDU activo, ±2 g, filtro de 50 Hz y autoincremento de dirección.

Expression	Type	Value
who	volatile uint8_t	0x3f
init_err	volatile uint8_t	0x0
r4	volatile uint8_t	0x6f
r5	volatile uint8_t	0xc0
r6	volatile uint8_t	0x10

Figura 2 *– Valores leídos en los registros.*

1.5.3). Leer los seis bytes de salida en un bucle y observar en el depurador (Live Expressions) que cambian de forma coherente al inclinar la placa.

En el bucle principal se leen los seis registros de salida (OUT_X_L a OUT_Z_H) en una sola transacción de 7 bytes. Cada eje se reconstruye como entero de 16 bits con signo a partir de su byte alto y su byte bajo: ax = (int16_t)((raw[1] << 8) | raw[0]).

Con la placa en reposo sobre la mesa, el eje z mide unas 17597 cuentas, que con la sensibilidad de 0,06 mg/LSB equivalen a ≈ 1056 mg ≈ 1,06 g, la gravedad. Los ejes x e y quedan próximos a cero. Al inclinar la placa, la gravedad se reparte entre los ejes, por ejemplo, al ponerla de canto, x pasa a medir ≈ 1,03 g y z se acerca a 0.

ax	volatile int16_t	418
ay	volatile int16_t	-89
az	volatile int16_t	17597

Figura 3 *– Placa sobre la mesa.*

ax	volatile int16_t	17134
ay	volatile int16_t	287
az	volatile int16_t	70

Figura 4 *– Placa inclinada.*

1.5.4). Si WHO_AM_I no dio 0x3F a la primera, documentar el síntoma y la causa (modo SPI, CS, byte de dirección, primer byte recibido...).

En la primera prueba que se realizó, *lis_init()* devolvía error (init_err = 1) y los registros de control conservaban sus valores por defecto (CTRL_REG4 = 0x07, CTRL_REG5 = 0x00), aunque la lectura de WHO_AM_I que se hace después sí devolvía 0x3F (Figura X). Es decir, solo fallaba la primera transacción SPI tras el arranque, que es precisamente la prueba de vida que hace *lis_init()*.

Expression	Type	Value
who	volatile uint8_t	0x3f
init_err	volatile uint8_t	0x1
r4	volatile uint8_t	0x7
r5	volatile uint8_t	0x0
r6	volatile uint8_t	0x10

Figura 5 *– Fallo en la primera prueba.*

Lo que ocurre es que el sensor no tiene reloj propio, lo que hace es contar los bits que recibe a partir de los flancos de la línea SCK, y solo mientras CS está a nivel bajo. Entonces, al inicio, SCK está bajo (SPI apagado) y CS está alto, por lo que el sensor no escucha y no ocurre nada. Pero cuando *lis_read* baja CS, el sensor empieza a escuchar mientras SCK sigue a nivel bajo. A continuación, se enciende el SPI y SCK sube a su nivel de reposo. Para el maestro esto no es una transmisión, solo la colocación del reloj en reposo; pero el sensor, que ya está escuchando, no puede distinguirlo de un flanco de reloj real y lo interpreta como un flanco de reloj, es decir, como un bit recibido. Cuando

empieza la transmisión real, el sensor ya ha contado un bit falso y todo lo que recibe le llega desplazado, por tanto entiende mal la dirección del registro y su respuesta no es 0x3F.

Para solucionarlo, se enciende el SPI antes de la primera transacción, con __HAL_SPI_ENABLE(&hspi1) y una breve espera (HAL_Delay(10)). Así SCK sube a su nivel de reposo cuando CS todavía está alto y el sensor no escucha. Cuando después se baja CS, SCK ya está en su sitio y no hay flanco falso.

```
/* USER CODE BEGIN 2 */
__HAL_SPI_ENABLE(&hspi1);
HAL_Delay(10);
cs_high();
init_err = lis_init();
lis_read(WHOAMI_REG, (uint8_t*)&who, 1);
lis_read(CTRL_REG4,  (uint8_t*)&r4, 1);
lis_read(CTRL_REG5,  (uint8_t*)&r5, 1);
lis_read(CTRL_REG6,  (uint8_t*)&r6, 1);
/* USER CODE END 2 */
```

Figura 6 *– Solución para el fallo de la primera prueba.*

PARTE B — *UN PLANIFICADOR HECHO A MANO: EL EJECUTIVO CÍCLICO.*

PASO B.1 — DE CUENTAS A UNIDADES FÍSICAS.

Cada eje se reconstruye como entero de 16 bits con signo a partir de sus dos bytes (Reto 1.2) y se convierte a miligravedades multiplicando por la sensibilidad del fondo de escala ±2 g, 0,06 mg/LSB:

```
4  static void convertir_mg(void)
5  {
6      ax_mg = ax * 0.06f;
7      ay_mg = ay * 0.06f;
8      az_mg = az * 0.06f;
9  }
```

Figura 7 *– Conversión de ejes a mg.*

Las variables en mg son de tipo float, y la constante se escribe con el sufijo f para que la operación se haga en precisión simple, la que soporta la FPU del Cortex-M4. Sin el sufijo, 0.06 sería un double y la multiplicación se haría por software.

PASO B.2 — EJEMPLO SENCILLO: UN EJECUTIVO CÍCLICO PARA EL MIV-RT.

Reto 1.6 — Ejecutivo cíclico: construirlo y romperlo.

1.6.1). Calcular el ciclo principal y dibujar la tabla de marcos (qué actividad va en cada marco). Implementarla en el Listado 1.2 junto con la conversión a mg.

Tarea	Periodo T (ms)
Leer Sensor (S)	10

Convertir a mg (C)	20
Latido (L)	50

Elegimos el ciclo secundario (marco) como el periodo más corto, en este caso los 10 ms de la tarea leer sensor. El ciclo principal se elige como el mínimo común múltiplo de los periodos, luego mcm(10, 20, 50) = 100 ms, así que contiene 10 marcos. La tabla de marcos queda de la siguiente manera:

Marco	0 (0ms)	1 (10ms)	2 (20ms)	3 (30ms)	4 (40ms)	5 (50ms)	6 (60ms)	7 (70ms)	8 (80ms)	9 (90ms)
S	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
C	✓		✓		✓		✓		✓	
L	✓					✓				

Para implementar la tabla se usa un contador de marco k, que va de 0 a 9 y vuelve a empezar en cada ciclo principal. En cada marco se decide qué actividades ejecutar según el valor de k: el sensor se lee siempre; la conversión a mg solo en los marcos pares (k % 2 == 0, es decir, cada 20 ms); y el LED de latido solo en los marcos 0 y 5 (k % 5 == 0, cada 50 ms).

```
uint32_t t0 = DWT->CYCCNT;
leer_sensor();
C_sensor_us = (DWT->CYCCNT - t0) / 168;

//convertir a mg
if (k % 2 == 0) {
    t0 = DWT->CYCCNT;
    convertir_mg();
    C_conv_us = (DWT->CYCCNT - t0) / 168;
}

//led verde
if (k % 5 == 0) {
    t0 = DWT->CYCCNT;
    latido();
    C_led_us = (DWT->CYCCNT - t0) / 168;
}
```

Figura 8 *- Implementación de la tabla de marcos.*

Cada marco empieza con una espera activa: el programa se queda en un bucle vacío hasta que llega la hora de inicio de ese marco. La hora del marco siguiente se obtiene sumando 10 ms a la hora prevista del marco actual (t_marco += MARCO_MS), no a la hora real a la que ha terminado. Así, si un marco se retrasa un poco, el retraso no se arrastra a los siguientes y la tabla se mantiene alineada con el tiempo real.

Al final de cada marco se comprueba si ya ha pasado la hora de inicio del siguiente. Si es así, el marco ha durado más de 10 ms (se ha desbordado) y se suma uno al contador overruns.

```
    t_marco = t_marco + MARCO_MS;
    if ((int32_t)(HAL_GetTick() - t_marco) > 0) overruns++;
    k = (k + 1) % N_MARCOS;
  /* USER CODE END 3 */
```

Figura 9 *- Cálculo del siguiente marco y detección de desbordamientos.*

1.6.2). Medir el periodo de los marcos y su jitter (PA8 o DWT) y el tiempo de cada actividad.

Los tiempos se han medido con el contador de ciclos DWT (DWT->CYCCNT, 168 ciclos por µs). El periodo del marco es de 10000 µs, con un máximo de 10000 µs y un mínimo de 9926 µs, lo que da un jitter de 74 µs, muy por debajo del margen de ±0,5 ms de RF-1. La lectura del sensor tarda 26 µs (un 0,26 % del marco), y la conversión a mg y el latido duran menos de 1 µs, además no se ha producido ningún desbordamiento.

periodo_us	volatile uint32_t	10000
periodo_max_us	volatile uint32_t	10000
periodo_min_us	volatile uint32_t	9926
periodo_max_us - pe	unsigned int	74
C_sensor_us	volatile uint32_t	26
overruns	volatile uint32_t	0
C_conv_us	volatile uint32_t	0
C_led_us	volatile uint32_t	0

Figura 10 - *Medidas del ejecutivo cíclico sin la actividad larga.*

1.6.3). Añadir, en un solo marco, una actividad «larga» de 15 ms de cómputo (una espera activa basta) y describir qué les ocurre a la lectura del sensor y al contador overruns.

Se ha añadido en el marco 3, después de la lectura del sensor, una actividad de 15 ms de espera activa. Como no cabe en un marco de 10 ms, ese marco se desborda en cada ciclo principal, pudiendo observarse en la figura 11 que tras unos 10 s de ejecución, overruns = 102, es decir, ocurre un desbordamiento aproximadamente cada 100 ms.

La lectura del sensor también se ve afectada. La lectura del marco 4, que debería hacerse a los 40 ms del ciclo, no puede empezar hasta que termina la actividad larga, hacia los 45 ms, de modo que ese periodo de lectura mide 15 000 µs. Además, como el marco 5 vuelve a empezar a su hora (50 ms) el periodo siguiente dura solo 4 999 µs. El jitter de la lectura pasa de 74 µs a 10 001 µs, muy por encima del margen de ±0,5 ms de RF-1.

periodo_us	volatile uint32_t	10000
periodo_max_us	volatile uint32_t	15000
periodo_min_us	volatile uint32_t	4999
periodo_max_us - pe	unsigned int	10001
C_sensor_us	volatile uint32_t	26
overruns	volatile uint32_t	102
C_conv_us	volatile uint32_t	0
C_led_us	volatile uint32_t	0

Figura 11 - *Medidas del ejecutivo cíclico con la actividad larga de 15 ms.*

1.6.4). ¿Cómo habría que reorganizar la tabla para que esa actividad cupiera sin romper el periodo de 10 ms? ¿Qué precio se paga? ¿Qué aportaría en cambio un planificador expulsivo con prioridades (Parte C)?

Según lo visto en teoría, para que esa actividad cupiera sin romper el periodo de 10ms, habría que dividirla de modo que cada parte quepa en los huecos libres de los marcos, por ejemplo, en 3 partes de 5ms en los marcos 1, 4 y 7, de forma que cada marco ocupa unos 5 ms y ninguno se desborda. El problema es que la actividad va a tardar más en completarse (porque se reparte a lo largo del ciclo principal) y la tabla estaría calculada específicamente para estos tiempos, y si, o bien una actividad pasa a durar más o se añade otra, o bien cambia algún periodo, hay que volver a rehacer todo a mano. Con un planificador expulsivo, en cambio, la actividad larga se va a escribir de una sola vez (sin divisiones), en una tarea de baja prioridad, y el sistema operativo la interrumpe solo cuando le toca a Sensor (mayor prioridad).

PARTE C — FREERTOS: LAS TAREAS DEL MODELO CON PRIORIDADES RM

PASO C.1 — ACTIVAR FREERTOS.

Se ha activado FreeRTOS desde CubeMX (Middleware → FreeRTOS) con la interfaz CMSIS_V2. En Config parameters se ha fijado configTICK_RATE_HZ = 1000, es decir, un tick de 1 ms, y configUSE_PREEMPTION = 1, para que el planificador sea expulsivo, cuando una tarea de mayor prioridad pasa a lista, desaloja inmediatamente a la que está en ejecución.

FreeRTOS usa el SysTick como reloj de su tick, así que la HAL ya no puede seguir usándolo para su base de tiempos (HAL_GetTick(), HAL_Delay() y los timeouts de los drivers). Por eso, en System Core → SYS → Timebase Source, se ha elegido el temporizador libre TIM6. CubeMX genera el archivo stm32f4xx_hal_timebase_tim.c, que programa TIM6 para interrumpir cada 1 ms, y en main.c la función que mantiene la base de tiempos de la HAL.

Esta función es imprescindible: si se elimina, el contador de la HAL deja de avanzar y cualquier HAL_Delay() se bloquea para siempre. El resto de los parámetros se han dejado por defecto, con heap_4 y configTOTAL_HEAP_SIZE = 15360 bytes, suficiente para las pilas de las cuatro tareas (4 × 1024 B) y las tareas internas del núcleo (Idle y Timer).

PASO C.2 — LAS TAREAS DEL MODELO.

Reto 1.7 — Prioridades y planificabilidad

1). Asignar las prioridades por RM y elegir sus niveles CMSIS. En el Hito 2 habrá que insertar dos tareas más por debajo de Sensor (Diagnóstico, de prioridad mínima, y otra de prueba, de prioridad intermedia): ¿deja hueco la asignación elegida sin alterar el orden de las cuatro?

Se han creado los cuatro hilos de la tabla de tareas del modelo desde la pestaña Tasks and Queues de CubeMX. Cada uno tiene su función de entrada, una pila de 256 palabras (1024 bytes) y la prioridad CMSIS justificada en el Reto 1.7:

TAREA	FUNCIÓN	T (ms)	PRIORIDAD CMSIS	TRABAJO EN PARTE C

Sensor	Task_sensor	10	osPriorityHigh	Leer los tres ejes del acel.
Proceso	Task_proceso	20	osPriorityAboveNormal	Conertir a mg
Seguridad	Task_seguridad	50	osPriorityNormal	Latido (LED verde)
Comms	Task_comms	100	osPriorityBelowNormal	Vacía hasta hito 3

CubeMX genera una estructura de atributos por tarea y la llamada que crea cada hilo, entre osKernelInitialize() y osKernelStart().

Las cuatro tareas siguen el mismo esquema. Guardan el instante de su primera activación y, en cada vuelta, hacen su trabajo, miden con el DWT cuánto ha tardado (su C) y se duermen hasta el siguiente instante absoluto con osDelayUntil.

Con Rate Monotonic, a menor periodo, mayor prioridad, así que el orden es Sensor (10 ms) > Proceso (20 ms) > Seguridad (50 ms) > Comms (100 ms). Se han elegido niveles CMSIS-RTOS2 con nombre, dejando un nivel entero entre cada tarea:

TAREA	T(ms)	Orden RM	Nivel CMSIS	Valor
Sensor	10	1	osPriorityHigh	40
Proceso	20	2	osPriorityAboveNormal	32
Seguridad	50	3	osPriorityNormal	24
Comms	100	4	osPriorityBelowNormal	16

La asignación sí deja hueco. Entre dos niveles con nombre hay siete subniveles (por ejemplo, osPriorityNormal1 … osPriorityNormal7, valores 25 a 31), y por debajo de Comms quedan libres osPriorityLow (8) y sus subniveles. Diagnóstico, de prioridad mínima, puede ir en osPriorityLow. La tarea de prueba, de prioridad intermedia, puede ir en un subnivel libre por debajo de Sensor, por ejemplo osPriorityNormal4 (28), entre Proceso y Seguridad. Ninguna de las dos obliga a cambiar el nivel de las cuatro tareas actuales.

2). Aplicar el test de Liu & Layland con los C estimados del modelo.

El test de Liu & Layland es una condición suficiente para RM: si la utilización total no supera la cota para n tareas, todas cumplen sus plazos. Para n = 4, la cota es 4 × (2^(1/4) − 1) = 0,757. Con los C estimados de la Tabla 5 del modelo de referencia:

Tarea	T(ms)	C estimado	C/T
Sensor	10	0,8	0,080
Proceso	20	1,5	0,075

Seguridad	50	0,3	0,006
Comms	100	2,0	0,020
Total			**U = 0,181**

U = 0,181 ≤ 0,757, así que, según el modelo, el conjunto es planificable con RM y con mucho margen.

3). Medir el C real de cada tarea (DWT o traza) y repetir el test. ¿Cambia la conclusión? Actualizar la tabla de tareas con los valores medidos.

Se ha medido el C de cada tarea con el DWT, con el sistema en marcha y sin cargas artificiales, y se ha tomado el máximo observado:

(x)= c_sensor_max_us	volatile uint32_t	25
(x)= c_proceso_max_us	volatile uint32_t	0
(x)= c_comms_max_us	volatile uint32_t	0
(x)= sensor_periodo_us	volatile uint32_t	9998
(x)= sensor_per_max_us	volatile uint32_t	10004
(x)= sensor_per_min_us	volatile uint32_t	9995
(x)= sensor_jitter_us	volatile uint32_t	9
(x)= sensor_R_max_ms	volatile uint32_t	0
(x)= sensor_perdidos	volatile uint32_t	0

Figura C.3 – Tiempos de cómputo y periodo de Sensor medidos en el sistema normal (Live Expressions).

Tarea	T (ms)	C medido	C/T
Sensor	10	25 µs	0,0025
Proceso	20	< 1 µs (el DWT marca 0)	< 0,0001

Seguridad	50	≤ 0,3 ms (no registrado; se acota con el C estimado)	≤ 0,006
Comms	100	0 µs (cuerpo vacío)	0
Total			**U ≤ 0,009**

Con los C medidos, la utilización baja a U ≤ 0,009 (menos de un 1 %). La conclusión no cambia: el sistema es planificable con RM, y con mucho más margen del previsto. Los C reales son mucho menores que los estimados. La lectura del sensor tarda 25 µs frente a los 0,8 ms del modelo, porque la transferencia SPI de 7 bytes a 5,25 MHz solo dura unos 10,7 µs (Reto 1.4). Proceso y Seguridad apenas calculan nada todavía. Con una utilización tan baja, cualquier asignación de prioridades funcionaría; por eso el Paso C.4 infla artificialmente el cómputo de Proceso.

La tabla de tareas actualizada queda así:

Tarea	T = D (ms)	Prioridad	Criticidad	C estimado (ms)	C medido (Parte C)
Sensor	10	osPriorityHigh	hard	0,8	25 µs
Proceso	20	osPriorityAboveNormal	hard/firm	1,5	< 1 µs
Seguridad	50	osPriorityNormal	soft	0,3	no registrado
Comms	100	osPriorityBelowNormal	firm	2,0	0 µs (vacía)

Paso C.3 — Periodicidad exacta.

Reto 1.8 — Periodicidad medida.

1). Medir periodo y *jitter* de Sensor con el Listado 1.3 tal cual, añadiendo dentro del bucle una carga artificial de _2 ms para que la diferencia se vea.

Para medir el periodo real de Sensor se lee el DWT al principio de cada activación; la diferencia con la lectura anterior es el periodo. Se guardan también su máximo y su mínimo, y el jitter es la diferencia entre ambos. Las dos primeras vueltas se descartan para no contar el arranque. La carga de 2 ms se añade con quemar_cpu_us(2000).

ax	volatile int16_t	97
ay	volatile int16_t	-2285
az	volatile int16_t	17301
uwTick	volatile uint32_t	118389
sensor_jitter_us	volatile uint32_t	1
sensor_periodo_us	volatile uint32_t	12000

Figura C.4 – Periodo y jitter de Sensor con osDelay(10) y carga de 2 ms.

Con osDelay(10) el periodo medido es de 12 000 µs, con un jitter de 1 µs. El periodo no es de 10 ms, sino de 12 ms: osDelay(10) cuenta los 10 ms desde que se le llama, es decir, después de los 2 ms de trabajo, así que el periodo real es T + C. El jitter es casi nulo porque la carga es siempre la misma, pero eso engaña: cada activación llega 2 ms más tarde que la anterior respecto al instante ideal, y ese error se acumula sin límite (a los 100 ms ya va 20 ms retrasada). Es la deriva que muestra la Figura 1.1 de la guía. Si la carga variase, el periodo variaría con ella.

2). Corregirlo con **osDelayUntil**, repetir la medida en las mismas condiciones y explicar la diferencia con la Figura 1.1.

La versión corregida calcula el instante absoluto de la siguiente activación a partir del anterior, no del momento en que termina el trabajo.

ax	volatile int16_t	120
ay	volatile int16_t	-2149
az	volatile int16_t	17292
uwTick	volatile uint32_t	9898
sensor_jitter_us	volatile uint32_t	1
sensor_periodo_us	volatile uint32_t	10000

Figura C.5 – Periodo y jitter de Sensor con osDelayUntil y carga de 2 ms.

Con osDelayUntil y la misma carga, el periodo medido es de 10 000 µs, con un jitter de 1 µs. La tarea despierta en los instantes 0, 10, 20, 30… ms, tarde lo que tarde su trabajo. El cómputo C solo retrasa el final de cada activación, nunca el comienzo de la siguiente, así que no hay deriva. Es el comportamiento de la última línea de la Figura 1.1.

3). Comprobar que se cumple RF-1 y retirar la carga artificial.

Con osDelayUntil el periodo de Sensor se mantiene en 10 ms incluso con la carga de 2 ms, dentro de los 10 ± 0,5 ms de RF-1. Después se ha retirado la carga y se ha dejado osDelayUntil en las cuatro tareas. En el sistema final, sin cargas (Figura C.3), el periodo medido es de 9 998 µs, con un máximo de 10 004 µs y un mínimo de 9 995 µs: un jitter de 9

µs, muy por debajo del margen de ±500 µs de RF-1. Ese jitter residual viene de la latencia del tick y del cambio de contexto, no del cómputo.

Paso C.4 — Ejemplo sencillo: sobrecarga y asignación de prioridades.

Reto 1.9 — Sobrecarga: RM frente a otra asignación

1). Con Proceso a C = 12 ms, calcular U y aplicar el test de Liu & Layland. Calcular con RTA el tiempo de respuesta de peor caso de Sensor y de Proceso en dos casos: (a) prioridades RM; (b) Proceso por encima de Sensor.

Con los C del modelo para el resto de tareas:

U = 0,8/10 + 12/20 + 0,3/50 + 2,0/100 = 0,080 + 0,600 + 0,006 + 0,020 = 0,706

U = 0,706 ≤ 0,757, así que, con prioridades RM, Liu & Layland garantiza que todas las tareas cumplen sus plazos, aunque con poco margen.

El análisis del tiempo de respuesta (RTA) calcula el peor caso de cada tarea sumando a su C la interferencia de las tareas de mayor prioridad. Se itera hasta que el valor no cambia.

Caso (a), prioridades RM (Sensor > Proceso):

Sensor no tiene tareas por encima: R_Sensor = C_Sensor = 0,8 ms ≤ D = 10 ms. Cumple.

Proceso sufre la interferencia de Sensor: R^0 = 12 + 0,8 = 12,8 ms; R^1 = 12 + ⌈12,8/10⌉ × 0,8 = 13,6 ms; R^2 = 12 + ⌈13,6/10⌉ × 0,8 = 13,6 ms. Converge en R_Proceso = 13,6 ms ≤ D = 20 ms. Cumple.

Caso (b), Proceso por encima de Sensor:

Proceso no tiene tareas por encima: R_Proceso = 12 ms ≤ 20 ms. Cumple.

Sensor sufre la interferencia de Proceso: R^0 = 0,8 + 12 = 12,8 ms; R^1 = 0,8 + ⌈12,8/20⌉ × 12 = 12,8 ms. Converge en R_Sensor = 12,8 ms > D = 10 ms. No cumple.

2). Instrumentar Sensor para medir su **tiempo de respuesta máximo** (desde su instante de activación hasta el final de su trabajo) y **contar los plazos perdidos**.

El tiempo de respuesta se mide desde el instante teórico de activación (t_act, el valor de t_sig antes de sumarle el periodo) hasta que Sensor termina su trabajo. Se usa el tick del núcleo, con resolución de 1 ms, y se cuenta como plazo perdido cada activación con R > 10 ms.

3). Ejecutar los dos casos y comparar medidas y predicciones. ¿Qué requisito del modelo se incumple en el caso (b)? ¿Qué concluye el test de Liu & Layland en cada caso y por qué no basta?

c_sensor_max_us	volatile uint32_t	25
c_proceso_max_us	volatile uint32_t	12040
c_comms_max_us	volatile uint32_t	0
sensor_periodo_us	volatile uint32_t	10001
sensor_per_max_us	volatile uint32_t	10001
sensor_per_min_us	volatile uint32_t	9998
sensor_jitter_us	volatile uint32_t	3
sensor_R_max_ms	volatile uint32_t	0
sensor_perdidos	volatile uint32_t	0

Figura C.6 – Caso (a), prioridades RM, con Proceso a C = 12 ms.

c_sensor_max_us	volatile uint32_t	25
c_proceso_max_us	volatile uint32_t	12001
c_comms_max_us	volatile uint32_t	0
sensor_periodo_us	volatile uint32_t	27
sensor_per_max_us	volatile uint32_t	19973
sensor_per_min_us	volatile uint32_t	26
sensor_jitter_us	volatile uint32_t	19947
sensor_R_max_ms	volatile uint32_t	10
sensor_perdidos	volatile uint32_t	0

Figura C.7 – Caso (b), Proceso por encima de Sensor.

Medida	Caso a	Caso b
C de Proceso medido	12 040 µs	12 001 µs
R de Sensor, RTA (peor caso)	0,8 ms	12,8 ms
R de Sensor medido (máx.)	0 ms	10 ms
Plazos perdidos contados	0	0
Periodo de Sensor mín. / máx.	9 998 / 10 001 µs	26 / 19 973 µs
Jitter de Sensor	3 µs	19 947 µs
¿Cumple RF-1?	Sí	No

En el caso (a) las medidas coinciden con el RTA. Sensor desaloja a Proceso en cuanto se activa, su R medido es 0 ms (termina dentro del mismo tick, en 25 µs), no pierde ningún plazo y su periodo sigue en 10 ms con 3 µs de jitter. Proceso, aun con 12 ms de cómputo, cabe en sus 20 ms.

En el caso (b) Sensor deja de ser periódico: su periodo alterna entre 26 µs y 19 973 µs, con un jitter de casi 20 ms. Es decir, cada 20 ms Sensor hace dos lecturas seguidas y luego ninguna. El R máximo medido es de 10 ms y no de los 12,8 ms del RTA, y el contador de plazos perdidos se queda en 0. Ambas cosas se explican por la fase con la que arrancan las tareas:

1. Al arrancar el planificador, Proceso es la tarea de mayor prioridad. Toma su instante inicial (t = 0) y ocupa la CPU 12 ms. Sensor no llega a ejecutarse hasta t = 12 ms, y es entonces cuando toma su propio instante inicial con osKernelGetTickCount(). Sus activaciones quedan así en 12, 22, 32, 42… ms, desfasadas 2 ms respecto a las de Proceso (0, 20, 40…).
2. La activación de Sensor de t = 22 ms llega mientras Proceso se ejecuta (20 a 32 ms), así que no puede leer hasta t ≈ 32,03 ms. Su R real es de unos 10,03 ms. El contador de ticks marca 32 − 22 = 10 ms, y como la condición es R > 10, no lo cuenta como perdido. En realidad sí pierde el plazo, por unos 30 µs, pero la resolución de 1 ms del tick no lo detecta.
3. La activación de t = 32 ms ya ha llegado cuando termina la anterior, así que osDelayUntil vuelve en seguida y Sensor lee otra vez a los 26 µs. Después no vuelve a leer hasta t ≈ 52 ms. De ahí los periodos de 26 µs y 19 973 µs.

El RTA da 12,8 ms porque calcula el peor caso posible, el instante crítico en que Sensor y Proceso se activan a la vez. En el ensayo las activaciones estaban desfasadas 2 ms, así que el retraso real fue de 12 − 2 ≈ 10 ms: menor que el peor caso, como debe ser, pero igualmente por encima del plazo. La medida no contradice el RTA: confirma que en el caso (b) Sensor no cumple. Con un medidor de R en microsegundos (DWT) o con la condición R ≥ 10, el contador habría registrado unos 50 plazos perdidos por segundo, uno por cada activación de Proceso.

Requisito incumplido en el caso (b): RF-1. Sensor debe leer cada 10 ± 0,5 ms, y el periodo medido varía entre 26 µs y casi 20 ms, un jitter de 19 947 µs, unas 40 veces el margen permitido. Además, Sensor termina después de su plazo en una de cada dos activaciones, lo que también incumple RNF-1 (ninguna tarea hard pierde su plazo).

Qué concluye Liu & Layland en cada caso: la utilización U = 0,706 es la misma en los dos casos, porque no depende de las prioridades. En el caso (a) el test es válido y garantiza que se cumplen los plazos, como se ha medido. En el caso (b) el test no es aplicable: la cota de Liu & Layland solo vale si las prioridades se asignan por Rate Monotonic, y aquí Proceso (T = 20 ms) está por encima de Sensor (T = 10 ms). Aplicado sin pensar, diría «planificable», y se equivocaría.

Por qué no basta: Liu & Layland solo mira la utilización total. No tiene en cuenta qué tarea desaloja a cuál ni el tiempo de respuesta de cada una, y además es una condición suficiente pero no necesaria: si U supera la cota, no permite concluir nada. El RTA, en cambio, es un análisis exacto para prioridades fijas: calcula el peor tiempo de respuesta de cada tarea con la asignación real y lo compara con su plazo. Es el que detecta el fallo del caso (b), y las medidas en placa lo confirman.

PARTE D — ÁNGULOS, SEGURIDAD Y LA PRIMERA CONDICIÓN DE CARRERA.

Paso D.1 — Pitch y roll.

Reto 1.10 — Ángulos y FPU

1). Implementar el cálculo en grados en la tarea Proceso y validarlo con al menos tres posiciones de referencia (horizontal, de canto e inclinación intermedia medida con un transportador o una aplicación de nivel). Estimar el error frente a RF-2.

```
static void calcular_angulos(void)
{
    pitch_grados = atan2f(ax_mg, sqrtf(ay_mg*ay_mg + az_mg*az_mg)) * 57.29578f;
    roll_grados = atan2f(ay_mg, az_mg) * 57.29578f;
    }
```

Figura 12 *– Cálculo de ángulos en grados.*

Los ángulos se han validado en tres posiciones de referencia (Figuras 13, 14, 15):

Posición	Referencia	Pitch	Roll	Error
Horizontal	0º/0º	1,44º	-0,43º	1,4º
Inclinada	pitch 2,7° / roll 34° (app de nivel)	2,84º	35,45º	0,14° / 1,45°
De canto	Pitch 90º	88,62º		1,38º

(x)= pitch_grados	volatile float	1.44143331
(x)= roll_grados	volatile float	-0.430909932

Figura 13 *– Posición Horizontal.*

(x)= pitch_grados	volatile float	2.84287143
(x)= roll_grados	volatile float	35.4494781

Figura 14 *– Posición inclinada.*

(x)= pitch_grados	volatile float	88.6174316
(x)= roll_grados	volatile float	31.3633041

Figura 15 *– Posición de canto.*

No tenemos en cuenta el roll en el caso de canto, ya que se calcula como roll = atan2(a_y, a_z), y con la placa de canto (pitch = 90°), toda la gravedad cae sobre el eje X, y entonces a_y ≈ 0 y a_z ≈ 0.

Como se puede observar en la tabla, el error máximo obtenido es de 1,45°, de modo que cae dentro del límite de RF-2 (≤ 2°).

2). Demostrar que se usa la FPU: medir el tiempo del cálculo con atan2f/sqrtf y con atan2/sqrt, y comparar en el desensamblado las instrucciones que genera cada versión.

Para demostrar que se usa la FPU, se ha calculado el pitch de dos formas y se ha medido cada una con el contador DWT: en precisión simple (atan2f, sqrtf) y en doble precisión (atan2, sqrt con operandos double).

t_float	volatile uint32_t	271
t_double	volatile uint32_t	4994

Figura 16 – Mediciones para float y double.

La versión double es unas 18 veces más lenta. Luego queda comprobado que en la versión float el compilador genera instrucciones de la FPU (vldr, vmul.f32, vadd.f32, vmov.f32, Figura 17) que trabajan sobre los registros de coma flotante, y los argumentos de sqrtf y atan2f se pasan en esos mismos registros. En la versión double no aparece ninguna instrucción de la FPU, cada float se convierte a double con software (Figura 18).

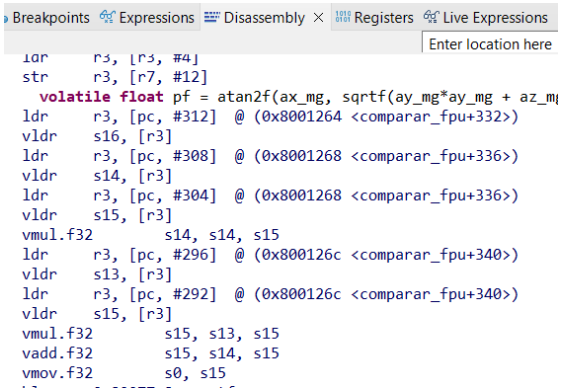


Figura 17 *– Versión float*

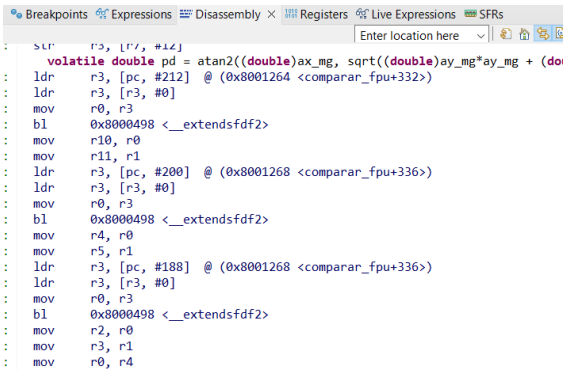


Figura 18 *– Version double*

Paso D.2 — La máquina de seguridad.

Reto 1.11 — Máquina de seguridad del modelo.

Se ha implementado la máquina de estados de la Figura 2.5 del modelo de referencia con dos switch, donde el primero decide el nuevo estado y el segundo muestra el estado en los LEDs

init_err	volatile uint8_t	0 '\0'
pitch_grados	volatile float	1.43642366
roll_grados	volatile float	-0.435887903

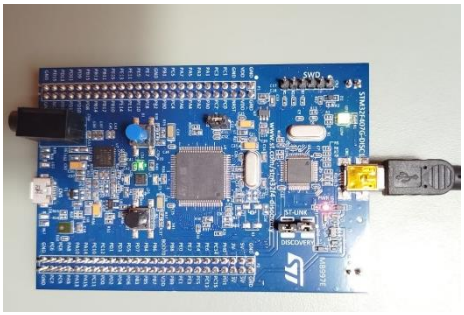


Figura 19 – *En posición Horizontal, está el LED verde encendido*

(x)= init_err	volatile uint8_t	0 '\0'
(x)= pitch_grados	volatile float	20.8719597
(x)= roll_grados	volatile float	-0.237856567

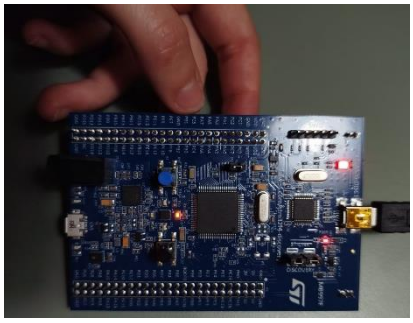


Figura 20 – *Con 20º está el LED naranja encendido*

(x)= init_err	volatile uint8_t	0 '\0'
(x)= pitch_grados	volatile float	45.2048302
(x)= roll_grados	volatile float	0.207507044

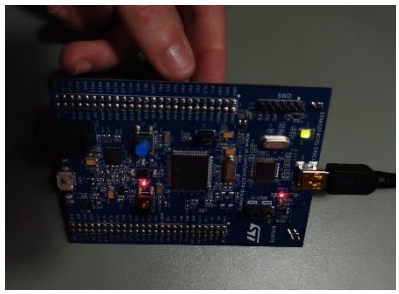


Figura 21 – *Con 45º está el LED rojo encendido*

La histéresis se demuestra a 14º. Se usa una goma de soporte para que la inclinación sea justo 14º, y comprobamos que, si se llega a 14º subiendo, el LED es verde, pero si se llega bajando, el LED es naranja.

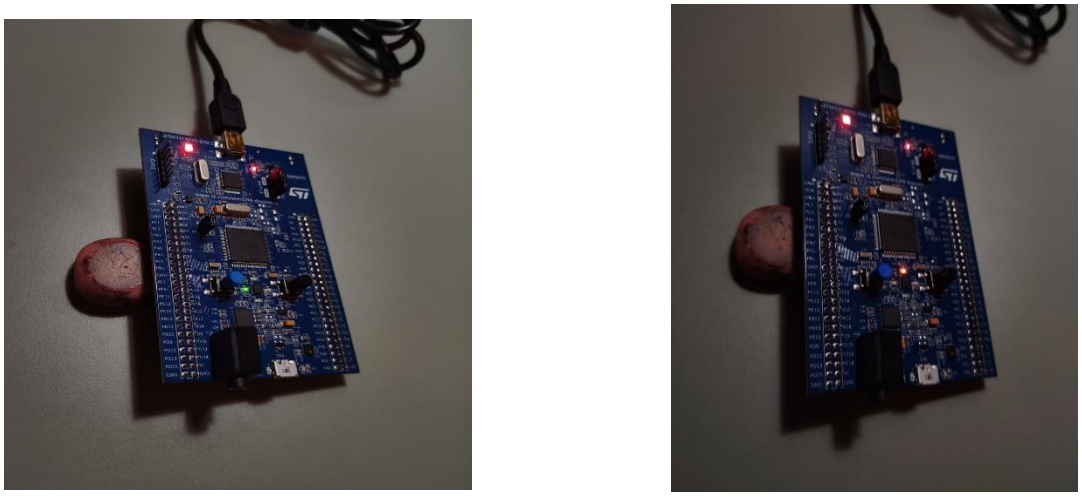


Figura 22 – *Comprobación de histéresis a 14º, se llega al LED verde subiendo (izquierda) y al naranja bajando (derecha).*

Retardo y RF-3. El retardo entre un cambio de inclinación y el cambio de LED depende de la cadena de tareas. Primero, la tarea Sensor lee la muestra (cada 10 ms), luego la tarea Proceso calcula los ángulos (cada 20 ms) y por último Seguridad decide el estado (cada 50 ms). En el peor de los casos, el cambio ocurre justo después de cada una de esas activaciones, y el retardo es de unos 10 + 20 + 50 = 80 ms, más los tiempos de respuesta de las tareas, que son del orden de microsegundos. Esto cumple el límite de retardo ≤ 100 ms de RF-3.

Paso D.3 — Ejemplo sencillo: una condición de carrera.

Reto 1.12 — Condición de carrera.

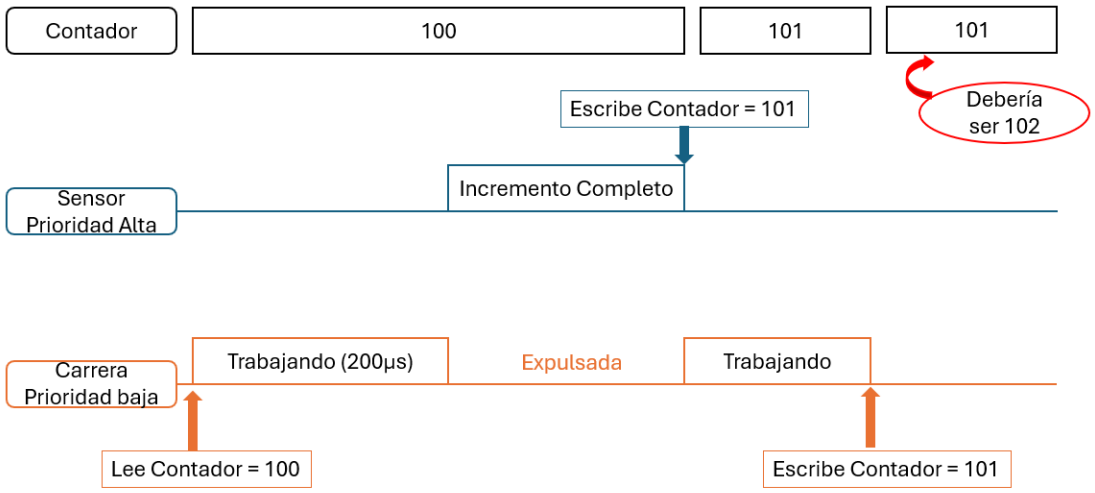
1). Crear Task_Carrera (bucle continuo: incrementa(); privado_carrera++;) y añadir a Sensor incrementa(); privado_sensor++;. Tras unos segundos, comparar contador con privado_sensor + privado_carrera. Explicar con un cronograma cómo se pierde un incremento, de qué tarea es el incremento perdido y por qué la pérdida es aproximadamente proporcional al número de activaciones de Sensor.

Se ha creado Task_Carrera con prioridad mínima (osPriorityLow) y un bucle continuo sin esperas que llama a incrementa() e incrementa su contador privado. La tarea Sensor hace lo mismo una vez por activación. La función incrementa() lee el contador compartido, trabaja 200 µs y escribe el valor leído más uno. Como cada tarea lleva su propio contador privado, que solo modifica ella, el valor correcto del compartido debería ser la suma de los dos privados. Tras unos 10s de ejecución, se obtienen los tiempos mostrados en la figura 23:

contador	volatile uint32_t	53209	0x200001e8
privado_sensor	volatile uint32_t	1098	0x200001ec
privado_carrera	volatile uint32_t	53208	0x200001f0
privado_sensor + privado_carrera - contador	unsigned int	1097	0x200001eb
sensor_per_max_us	volatile uint32_t	10004	0x200001bc
sensor_per_min_us	volatile uint32_t	9995	0x20000004
sensor_jitter_us	volatile uint32_t	9	0x200001c0

Figura 23

Se han perdido 1097 de los 1098 incrementos de Sensor. Lo que sucede es que, la tarea Carrera se ejecuta continuamente, durando cada incremento 0,2ms. Cuando la tarea Sensor despierta cada 10 ms, como tiene mayor prioridad, se ejecuta y el planificador expulsa a Carrera hasta que Sensor termine. Si por ejemplo, el contador estaba a 100, cuando llega Sensor, esta se ejecuta y realiza un incremento, de manera que contador queda en 101. Pero Carrera, al reanudarse, todavía tiene el valor que había leído antes, 100, de forma que escribe 100 + 1 = 101. Como ha habido dos incrementos el contador debería valer 102, pero vale 101, porque se ha perdido el incremento de Sensor. El siguiente cronograma muestra cómo se pierde un incremento:



2). Proteger la sección crítica con taskENTER_CRITICAL()/taskEXIT_CRITICAL() y comprobar que ya no se pierden incrementos. Medir el *jitter* de Sensor con y sin la protección: ¿qué precio se paga? ¿Por qué no es buena idea usar este mecanismo para secciones críticas largas? (El Tema 4 (II) y el Hito 2 dan la alternativa.)

Se ha protegido el cuerpo de incrementa() con taskENTER_CRITICAL() y taskEXIT_CRITICAL(). Estas llamadas desactivan las interrupciones, de modo que el planificador no puede cambiar de tarea hasta que termina el incremento. Con la protección, el contador coincide exactamente con la suma de los privados:

contador	volatile uint32_t	70358	0x200001e8
privado_sensor	volatile uint32_t	1430	0x200001ec
privado_carrera	volatile uint32_t	68928	0x200001f0
privado_sensor + privado_carrera - contador	unsigned int	0	0x1ffffc56
sensor_per_max_us	volatile uint32_t	10193	0x200001bc
sensor_per_min_us	volatile uint32_t	9935	0x20000004
sensor_jitter_us	volatile uint32_t	258	0x200001c0

Figura 24

El precio es el aumento del jitter de Sensor, que pasa de 9 µs a 258 µs. Si el tick que debe despertar a Sensor llega mientras Task_Carrera está dentro de la sección crítica, la interrupción queda pendiente hasta que Task_Carrera termina, hasta 200 µs después, y Sensor se activa con retraso. Como osDelayUntil no acumula ese retraso, la activación siguiente vuelve a su hora y el periodo entre ambas es más corto: por eso el periodo oscila por encima y por debajo de 10 ms.

Así, una tarea de prioridad mínima retrasa a la de prioridad máxima. Ese retraso es un tiempo de bloqueo B que hay que sumar en el análisis de tiempo de respuesta: R = C + B. En este caso, R_Sensor ≈ 26 + 200 µs, muy por debajo del plazo de 10 ms, y el jitter sigue dentro del margen de RF-1 (±0,5 ms).

Este mecanismo no es adecuado para secciones críticas largas. Bloquea a todo el sistema, no solo a las tareas que comparten el recurso. El bloqueo crece con la duración de la sección crítica y empeora el tiempo de respuesta de tareas que no tienen relación con ella. Para secciones críticas largas se usa un mutex con herencia de prioridad, que solo bloquea a las tareas que usan el recurso.

3). Aplicarlo al MIV-RT: Sensor publica tres ejes en tres variables globales. ¿Es atómica cada escritura en un Cortex-M4? ¿Y el trío? ¿Puede Proceso combinar ejes de dos muestras distintas? Decidir si importa y, si importa, protegerlo.

En el MIV-RT, la tarea Sensor publica los tres ejes en tres variables globales (ax, ay y az), que después lee la tarea Proceso. Por un lado, cada escritura por separado sí es atómica, ya que el Cortex-M4 es un procesador de 32 bits, por lo que puede escribir en memoria hasta 32 bits de una sola vez, y las interrupciones solo pueden ocurrir entre una instrucción y otra, nunca a mitad. Como cada eje es un int16_t, se guarda con una única instrucción, esto no pasaría con una variable de 64 bits, como un double, ya que se escribiría en dos partes y podría producirse una interrupción entre ellas.

Sin embargo, el trío no es atómico, ya que son tres instrucciones distintas y entre ellas sí puede colarse otra tarea. Es decir, aunque cada eje se escribe de una vez, el conjunto de los tres puede quedar a medias.

¿Puede entonces Proceso mezclar ejes de dos muestras distintas? Para que esto ocurra, Sensor tendría que interrumpir a Proceso justo mientras está leyendo los tres ejes. Con la configuración actual esto no pasa, como los periodos son 10 y 20 ms, cuando ambas tareas se activan a la vez se ejecuta primero Sensor, que tiene más prioridad, y después Proceso, que termina en unos pocos microsegundos, mucho antes de que Sensor vuelva a activarse. El problema es que esto depende de los tiempos. Si Proceso tardara más, como en el ejemplo de sobrecarga del Reto 1.9 con 12 ms, Sensor sí la interrumpiría a mitad, y podría hacerlo justo mientras lee los ejes. En ese caso, Proceso podría quedarse con el eje X de una muestra y los ejes Y y Z de la siguiente.

Por eso se ha decidido que sí importa, un vector mezclado no corresponde a ninguna posición real de la placa y podría dar un ángulo incorrecto y, con él, un cambio de estado de seguridad falso. Además, protegerlo es muy sencillo y apenas tiene coste:

```
static void convertir_mg(void)
{
    taskENTER_CRITICAL();
    ax_mg = ax * 0.06f;
    ay_mg = ay * 0.06f;
    az_mg = az * 0.06f;
    taskEXIT_CRITICAL();

}
```

Figura 25

PARTE E — VIBRACIÓN, INTEGRACIÓN Y ENTREGA.

Paso E.1 — Índice de vibración (RF-6).

Reto 1.13 — Índice de vibración.

1). Implementar V en Proceso y el LED azul con histéresis. Elegir W para que se cumpla el retardo de RF-6 y justificarlo. ¿Por qué el modelo usa el módulo de a y no un eje? Comprobarlo: inclinar despacio la placa no debe encender el LED azul.

Según el modelo de referencia, V es la desviación típica del módulo de la aceleración en una ventana de W muestras:

$$ak = \sqrt{ax^2 + ay^2 + az^2}$$

$$V = \sqrt{\frac{1}{W}\sum_{k1}^{W}(ak - \overline{a})^2}$$

Sensor publica los tres ejes y su número de secuencia dentro de una sección crítica, y Proceso los copia de la misma forma. Así Proceso nunca mezcla ejes de dos lecturas distintas (el problema del Reto 1.12). La transacción SPI queda fuera de la sección crítica para no bloquear el sistema mientras dura.

La ventana se define sobre los números de secuencia de Sensor, las muestras 1 a 10 forman la ventana 0, las 11 a 20 la ventana 1, y así sucesivamente. Proceso guarda |a| de cada muestra nueva que recibe. Cuando llega una muestra de la ventana siguiente, cierra la anterior: calcula V con las muestras que ha recogido y aplica la histéresis al LED azul (LD6, PD15).

La histéresis evita que el LED parpadee cuando V ronda el umbral: se enciende al alcanzar VON = 30 mg y no se apaga hasta bajar de VOFF = 20 mg.

Se ha elegido W = 10 muestras de Sensor, es decir, una ventana de 10 × 10 ms = 100 ms. RF-6 pide que el LED azul se encienda con un retardo ≤ 1 s desde que empieza la vibración. En el peor caso, la vibración empieza justo después de abrirse una ventana: hay que esperar a que esa ventana termine (100 ms) y a que Proceso la cierre en su siguiente activación (como mucho 20 ms más). El retardo máximo es de unos 120 ms, muy por debajo de 1 s. Una ventana más corta daría una V poco fiable, con muy pocas muestras para la desviación típica. Una mucho más larga promediaría golpes breves y retrasaría la respuesta sin necesidad. Con 100 ms se usa poca memoria (un vector de 10 float) y un golpe de unas decenas de ms ocupa una parte importante de la ventana.

Por qué el módulo y no un eje. Con la placa en reposo, el acelerómetro solo mide la gravedad, así que |a| vale siempre unos 1000 mg, sea cual sea la inclinación. Al inclinar la placa, la gravedad se reparte de otra forma entre los ejes: cada eje cambia mucho (hasta ±1000 mg), pero el módulo no. Si V se calculara sobre un eje, una inclinación lenta haría variar ese eje dentro de la ventana y se confundiría con una vibración. Con el módulo, V solo crece cuando hay aceleraciones reales además de la gravedad, que es lo que producen los golpes y las sacudidas. Así, el indicador de vibración es independiente de la orientación, que ya vigila la máquina de seguridad.

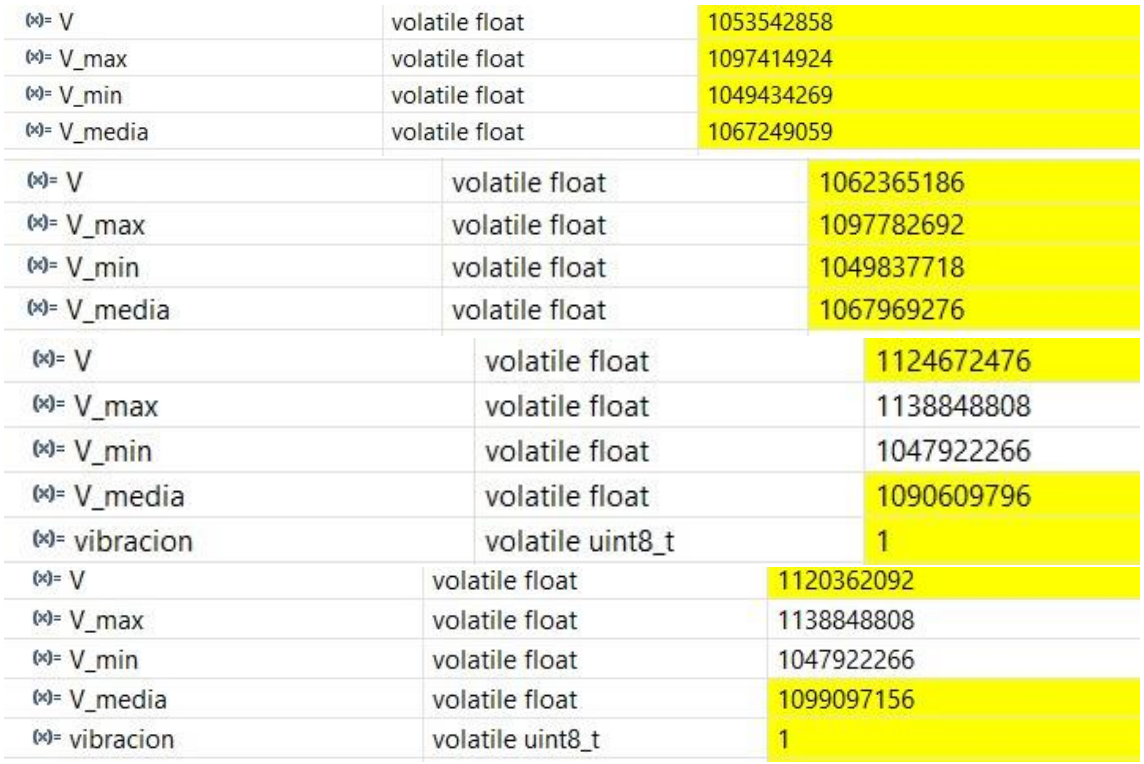
Comprobación. Al inclinar la placa despacio hasta unos 45° y volver, los LEDs de inclinación cambian de verde a naranja y rojo, pero el LED azul no se enciende. En la captura de esa prueba, V = 1,97 mg y vibracion = 0, del mismo orden que el ruido con la placa quieta (Reto 1.13.2).

(x)= V	volatile float	1073478755
(x)= V_max	volatile float	1138848808
(x)= V_min	volatile float	1047922266
(x)= V_media	volatile float	1090068562
(x)= vibracion	volatile uint8_t	0

Figura E.1 – Inclinación lenta de la placa: V del orden del ruido y LED azul apagado.

2). Caracterizar V en placa: con la placa quieta sobre la mesa (el ruido del sensor) y con al menos dos excitaciones reproducibles (golpes en la mesa, a giarla con la mano…). Con esas medidas, elegir Von y Voff con un margen justificado.

Para caracterizar V se añadieron al código unas estadísticas desde el último reinicio: V_max, V_min, V_media y el número de ventanas con el LED azul encendido. Se reinician con el botón de usuario de la placa (B1, PA0), que se lee en la tarea Seguridad. Cada medida empieza pulsando el botón, y las capturas se toman con el programa en marcha, sin pausar el depurador.



Figuras E.2 a E.5 – V con la placa quieta, inclinada despacio, con golpes en la mesa y agitada con la mano.

Ensayo	V en la captura (mg)	V_media (mg)	V_max (mg)	V_min (mg)	LED azul
Quieta, 20 s (1)	0,40	1,23	14,58	0,28	—
Quieta, 20 s (2)	0,82	1,31	14,93	0,29	—
Quieta, 60 s (603 ventanas)	1,22	1,48	93,91	0,14	1 ventana de 603
Inclinar despacio	1,97	—	—	—	Apagado
Golpes en la mesa	137,1	—	—	—	Encendido
Agitar con la mano	99,7	—	450,9	—	Encendido

Las capturas de inclinación, golpes y agitación se tomaron seguidas, sin reiniciar las estadísticas entre ellas. Por eso, en esos tres ensayos solo es fiable la V del momento de la captura, y el máximo de 450,9 mg corresponde al conjunto de las excitaciones. En un

segundo intento de inclinación se registró V = 138 mg: el movimiento no fue lo bastante lento y la placa recibió una sacudida. No se ha usado para elegir los umbrales.

Resultados.

- **Placa quieta (ruido del sensor):** V vale de media entre 1,2 y 1,5 mg, con picos aislados de unos 15 mg. En la medida de 60 s aparece un pico de 94 mg y una sola ventana de 603 con el LED encendido. Lo más probable es que se deba a la pulsación del botón con que empieza la medida, que mueve la placa.
- **Inclinación lenta:** V = 2 mg, del orden del ruido. El LED azul no se enciende, como se esperaba (Reto 1.13.1).
- **Golpes y agitación:** V sube a 100–140 mg en el momento de la captura, con picos de hasta 451 mg, y el LED azul se enciende.

Elección de los umbrales. Hay casi dos órdenes de magnitud entre el ruido (≈ 1,5 mg de media) y las excitaciones (≥ 100 mg), así que hay margen de sobra. Se mantienen VON = 30 mg y VOFF = 20 mg:

- **VON = 30 mg** es el doble de los picos de ruido con la placa quieta (≈ 15 mg), así que el ruido no enciende el LED. A la vez, es más de 3 veces menor que la V de los golpes y la agitación más suaves medidos (≈ 100 mg), así que detecta las excitaciones con margen.
- **VOFF = 20 mg** queda por encima de los picos de ruido, así que el LED se apaga en cuanto la placa vuelve a estar quieta. La banda de histéresis de 10 mg evita que el LED parpadee si la vibración ronda el umbral.

3). Añadir un número de secuencia a cada muestra en Sensor y contar desde Proceso cuántas muestras de cada ventana se usan de verdad. Explicar el resultado con los periodos y las prioridades del Reto 1.7. ¿Qué frecuencias de vibración puede ver aún V con las muestras que llegan? RF-6 pide la ventana completa: se resuelve en el Hito 2.

El número de secuencia seq_sensor se incrementa en cada lectura de Sensor, junto con los ejes y dentro de la misma sección crítica (código del Reto 1.13.1). Proceso compara la secuencia de cada muestra con la anterior que ha visto: si salta en más de 1, las muestras intermedias se han perdido; si es la misma, la muestra está repetida.

(x)= V	volatile float	1067202652
(x)= V_max	volatile float	1119605612
(x)= V_min	volatile float	1041051465
(x)= V_media	volatile float	1069402182
(x)= vibracion	volatile uint8_t	0
(x)= ventanas_medidas	volatile uint32_t	603
(x)= ventanas_con_vibracion	volatile uint32_t	1
(x)= usadas_ultima_ventana	volatile uint32_t	5
(x)= perdidas_ultima_ventana	volatile uint32_t	5
(x)= muestras_usadas	volatile uint32_t	3017
(x)= muestras_perdidas	volatile uint32_t	3017
(x)= muestras_repetidas	volatile uint32_t	0
(x)= c_sensor_max_us	volatile uint32_t	27
(x)= c_proceso_max_us	volatile uint32_t	10
(x)= c_comms_max_us	volatile uint32_t	0
(x)= sensor_periodo_us	volatile uint32_t	10001
(x)= sensor_per_max_us	volatile uint32_t	10004
(x)= sensor_per_min_us	volatile uint32_t	9995
(x)= sensor_jitter_us	volatile uint32_t	9
(x)= sensor_R_max_ms	volatile uint32_t	0
(x)= sensor_perdidos	volatile uint32_t	0
(x)= medida_n	volatile uint32_t	9

Figura E.6 – Contabilidad de muestras y tiempos tras 60 s con la placa quieta.

Variable	Valor medido
usadas_ultima_ventana	5 de 10
perdidas_ultima_ventana	5 de 10
muestras_usadas (60 s)	3 017
muestras_perdidas (60 s)	3 017
muestras_repetidas	0

De cada ventana de 10 muestras de Sensor, Proceso solo usa 5 y pierde las otras 5: exactamente una de cada dos. Se explica con los periodos del Reto 1.7. Sensor (T = 10 ms) produce una muestra nueva cada 10 ms, pero Proceso (T = 20 ms) solo lee la variable compartida cada 20 ms. Entre dos lecturas de Proceso, Sensor ha escrito dos veces, y la segunda escritura machaca a la primera, que no llega a usarse.

Las prioridades explican que el patrón sea tan regular, sin ninguna muestra repetida. Cada 20 ms, Sensor y Proceso se activan en el mismo tick, y como Sensor tiene más prioridad, siempre lee antes de que Proceso copie. Proceso usa siempre la muestra recién leída: las

de secuencia par se usan y las impares se pierden. Las 3 017 muestras usadas en unos 60 s confirman el ritmo de 50 muestras por segundo.

Frecuencias que puede ver V. Proceso trabaja en la práctica con 50 muestras por segundo, una cada 20 ms. Por el criterio de Nyquist, V solo puede representar correctamente vibraciones de hasta 25 Hz. Las vibraciones entre 25 Hz y 50 Hz (el ancho de banda configurado en el sensor, CTRL_REG5) siguen llegando, pero aparecen con aliasing, como si fueran de menor frecuencia. V las seguirá detectando como vibración, aunque con un valor menos fiable. Con las 100 muestras por segundo de Sensor se podría llegar a 50 Hz. RF-6 pide la ventana completa, sin perder muestras; eso requiere un mecanismo que lleve cada muestra de Sensor a Proceso, como una cola, y se resuelve en el Hito 2.

4). Medir el C de Proceso con el cálculo de V y rehacer el test de planificabilidad (RNF-1). ¿Cambia la conclusión del Reto 1.7?

Para esta medida se ha retirado la llamada a comparar_fpu() del Reto 1.10: calculaba un atan2 en double por software e inflaba artificialmente el C de Proceso. Ahora c_proceso_max_us solo incluye convertir_mg(), calcular_angulos() y calcular_vibracion(). Con el sistema completo y los ejemplos de sobrecarga y de carrera desactivados, se ha medido durante 60 s (Figura E.6):

Tarea	T (ms)	C medido	C/T
Sensor	10	27 µs	0,0027
Proceso (con ángulos y V)	20	10 µs	0,0005
Seguridad	50	≤ 0,3 ms (no registrado; se acota con el C estimado)	≤ 0,006
Comms	100	0 µs (vacía)	0
Total			**U ≤ 0,009**

El cálculo de V eleva el C de Proceso de menos de 1 µs (Parte C) a 10 µs: la raíz cuadrada, la media y la desviación típica de 5 valores en float con la FPU, más los dos atan2f de los ángulos. Aun así, U ≤ 0,009 queda muy lejos de la cota de Liu & Layland para 4 tareas (0,757). La conclusión del Reto 1.7 no cambia: el sistema es planificable con RM y se cumple RNF-1.

Las medidas lo confirman. Durante los 60 s el tiempo de respuesta máximo de Sensor fue de 0 ms (termina dentro del mismo tick), no se perdió ningún plazo (sensor_perdidos = 0) y el periodo de Sensor se mantuvo entre 9 995 y 10 004 µs, con 9 µs de jitter.

Paso E.2 — Integración.

Con los ejemplos de sobrecarga y de carrera **desactivadosB**(**#define** a 0), el sistema debe quedar como lo describe el modelo para este hito: Sensor, Proceso y Seguridad con sus periodos y prioridades, y Comms creada pero vacía. La sesión se dedica a integrar, repetir las medidas finales y cerrar la memoria. Es también el **punto de control**: el profesor revisa con cada grupo el estado del sistema antes de empezar el Hito 2.

Punto de verificación

- El periodo de Sensor es 10 ± 0;5 ms (RF-1) con el sistema completo.
- Pitch y roll coinciden con la inclinación real (RF-2).
- Los LEDs siguen la máquina de estados del modelo y no oscilan en los umbrales
- (RF-3); un fallo del sensor en el arranque lleva a FALLO (RF-4).
- El LED azul se enciende al golpear o agitar la placa y no al inclinarla despacio
- (RF-6, salvo la ventana completa).
- La tabla de tareas tiene los C medidos y el test de planificabilidad está hecho
- con ellos (RNF-1)

El sistema queda como lo describe el modelo para este hito. Sensor, Proceso y Seguridad funcionan con sus periodos (10, 20 y 50 ms) y sus prioridades RM. Comms está creada pero vacía. La tarea Carrera del Reto 1.12 se sigue creando desde el código generado por CubeMX, pero con EJEMPLO_CARRERA = 0 termina en cuanto arranca (osThreadExit()). Para probar RF-4 se usa PRUEBA_FALLO = 1, que fuerza init_err = 1 tras la inicialización del sensor, como si este no respondiera.

Punto de verificación

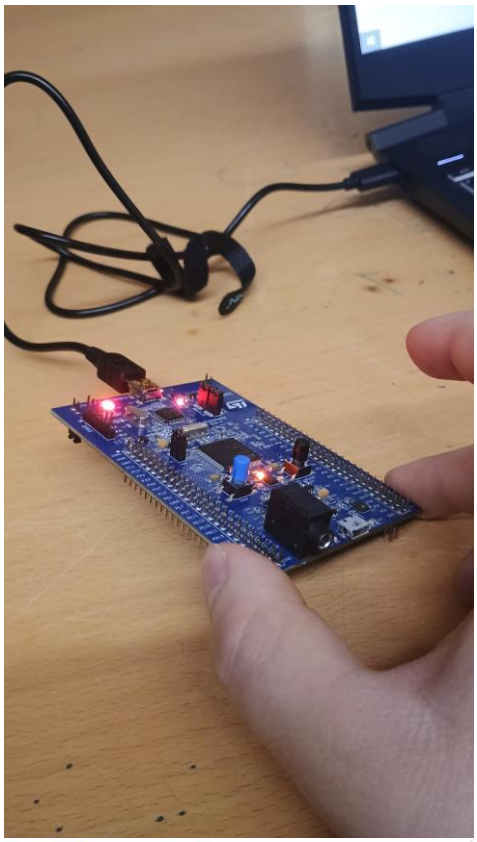
Punto	Resultado medido	¿Cumple?
Periodo de Sensor 10 ± 0,5 ms con el sistema completo (RF-1)	10 001 µs; mín. 9 995 µs, máx. 10 004 µs; jitter 9 µs (60 s)	Sí
Pitch y roll coinciden con la inclinación real (RF-2)	Placa plana: pitch = 0,76°, roll = −0,56°, error < 1° (≤ 2°)	Sí

LEDs según la máquina de estados, sin oscilar en los umbrales (RF-3)	0,8° → verde; 22,7° → naranja; 35,5° → rojo. Histéresis de 2° (15/13° y 30/28°)	Sí
Fallo del sensor en el arranque → FALLO (RF-4)	Con PRUEBA_FALLO = 1: init_err = 1 y LED rojo parpadeando	Sí
LED azul al golpear o agitar, no al inclinar despacio (RF-6)	Golpes V = 137 mg y agitación V = 100 mg → encendido; inclinación lenta V = 2 mg → apagado	Sí (salvo la ventana completa, Hito 2)
Tabla de tareas con C medidos y test de planificabilidad (RNF-1)	U ≤ 0,009 ≤ 0,757; 0 plazos perdidos y R de Sensor = 0 ms	Sí



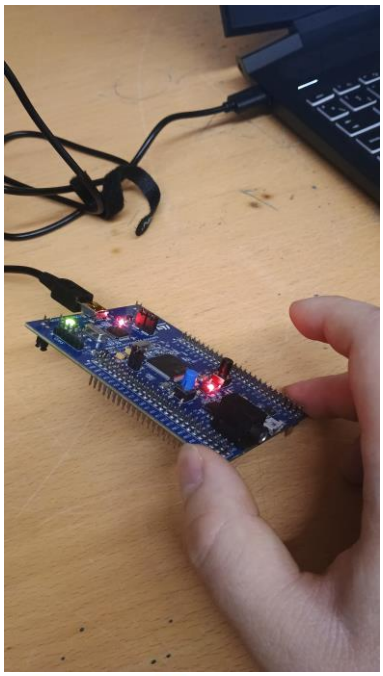
pitch_grados	volatile float	0.763862491
roll_grados	volatile float	-0.555873513

Figura E.7 – Pitch y roll con la placa plana (LED verde).



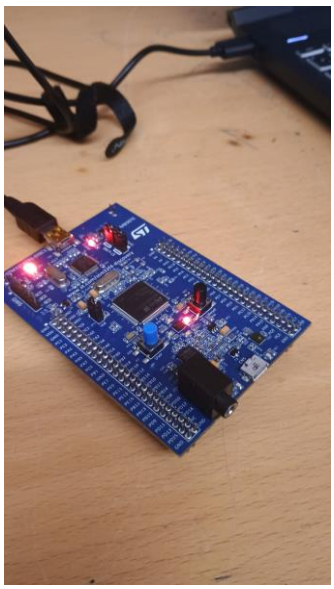
(x)= pitch_grados	volatile float	22.6219521
(x)= roll_grados	volatile float	-0.141566887

Figura E.8 – Placa inclinada unos 20° (pitch = 22,6°): LED naranja.



(x)= pitch_grados	volatile float	35.5255127
(x)= roll_grados	volatile float	-0.0682187825

Figura E.9 – Placa inclinada unos 35° (pitch = 35,5°): LED rojo.



(x)= init_err	volatile uint8_t	1

Figura E.10 – Prueba de RF-4: init_err = 1 y LED rojo parpadeando (estado FALLO).

Los valores de pitch con la placa inclinada se corresponden con las inclinaciones aproximadas a las que se colocó la placa (unos 20° y 35°), y en cada caso el LED encendido es el que fija la máquina de estados del modelo. El error de RF-2 solo se ha medido de forma estricta en la posición de referencia horizontal, donde los dos ángulos deben ser 0° y se obtiene un error menor de 1°.